Object Oriented Programming (Lab)

BSIT - Fall 2024

Dr. Muhammad Farooq

The objective of this lab is to:
To understand and utilize the concept of arrays (1D and 2D) along with the previous concepts.

ALERT!

1. This is an individual lab. You are **strictly NOT allowed to collaborate** with others, share screens, or communicate answers in any form.
2. Use of **AI tools (e.g., ChatGPT, Copilot, etc.) is strictly prohibited**. Any AI-generated content will be treated as academic dishonesty.
3. Anyone caught in the act of **cheating would be awarded a 0** in this Lab.
4. MAKE SURE TO **VALIDATE WHERE EVER YOU TAKE INPUT** FROM THE USER< OTHERWISE MARKS SHALL BE DEDUCTED.

Lab - 5

Task 01 (5 marks)

In a sales record, an element is called a *leader* if it is greater than all elements to its right. Write a program using **dynamic arrays and pointers** to find all leader elements.

Input:
Enter number of sales: 6
Sales: 16 17 4 3 5 2
Output:
Leader elements: 17 5 2

Task 02 (5 marks)

A store manager wants to keep track of daily sales. Write a program that asks the user for the number of days, takes daily sales input, writes them to a file named **sales.txt**, then reads the file to calculate and display the total and average sales.

Input:
Enter number of days: 3
Enter sales for day 1: 2500
Enter sales for day 2: 3200
Enter sales for day 3: 2800
Output:
Sales data from file:

Day 1: 2500
Day 2: 3200
Day 3: 2800

Total Sales: 8500
Average Sales: 2833.33

Task 03 (5 marks)

3. An array acts like a maze — each element represents the **index of the next element** to jump to.Starting at index 0, follow the path by using pointer arithmetic until you either:

- Go outside the array (exit successfully), or
- Enter a loop (infinite cycle).

Use only **pointers**, no indexing.

Input:	**Input:**
Enter number of elements: 6	Enter number of elements: 4
Enter elements: 2 4 1 3 6 0	Elements: 1 2 3 1
Output:	**Output:**
Path: 0 → 2 → 1 → 4 → 6	Path: 0 → 1 → 2 → 3 → 1 → 2 → ...
Exit reached successfully.	Loop detected.

Task 04 (5 marks)

You are given a **square matrix** whose size and elements are entered by the user.
Store it using **dynamic memory allocation** (`new`) and write a **function** to check whether it forms a *magic square* — i.e., the sum of each row, each column, and both diagonals is the same.

Write a function like:

```
bool isMagicSquare(int **matrix, int n);
```

Input:
Enter size: 3
Enter elements:
2 7 6
9 5 1
4 3 8
Output:
All sums = 15
It is a Magic Square.

Task 05 (4 marks)

In statistics, the **mode** of a set of values is the value that occurs most often or with the greatest frequency.
Write a function that accepts as arguments the following:
A) An array of integers
B) An integer that indicates the number of elements in the array.
The function should determine the **mode** of the array. That is, it should determine which value in the array occurs most often. The **mode** is the value the function should return. If the array has no **mode** (none of the values occur more than once), the function should return −1. [Assume the array will always contain non negative values].

Write a function like:

int findMode(int *arr, int size);

Input:

Array = [2, 3, 4, 2, 6, 2, 8]

Size = 7

Output:

Mode = 2

Task 06 (4 marks)

The task is to write a program that takes an input string from the user and calculates the **frequency** of each letter in the string. The program should only **count lowercase letters** and ignore any other characters, such as spaces or punctuation. The program must validate that the user inputs only lowercase letters. If the input contains any character that is not a **lowercase** letter, the program should display an error message and prompt the user to input a valid string. After validating the input, the program will output the **frequency** of each letter in the string.

Write a function like:

void countFrequency(const string &str);

Input:

oop lab

Output:

o = 2
p = 1

l = 1
a = 1
b = 1

Task 07 (4 marks)

John is using **a dynamically allocated array** to store the **marks** of his students. However, the array runs out of space as he adds more data. To overcome this issue, you are tasked with writing a function named **resize Array** that doubles the capacity of the array dynamically. To expand the array implement a **function resizeArray** that will take an array and its size as input. It should return a new array of double size, the first half of array should contain the elements of the original array and second half should contain two times the values of the input array.

Your function should have the following prototype:

int* resizeArray (int* inArray, const int size);

Input:

Enter size of the array: 4
Enter elements of array: 1 2 3 4

Output:

Resized array: 1, 2, 3, 4, 2, 4, 6, 8

Task 08 (4 marks)

Write a program that:
- Asks for the number of students and subjects.
- Takes marks for each student in each subject.
- Writes the data to `marks.txt` in tabular form.
- Reads the file and computes:
  - Average marks per student.
  - Subject-wise average.
- Displays both on screen.

Write a function like:

void processStudentMarks(int numStudents, int numSubjects);

Input:

Enter number of students: 3
Enter number of subjects: 3

Enter marks for Student 1: 80 75 90
Enter marks for Student 2: 60 70 65

Enter marks for Student 3: 95 85 80

Contents of marks.txt:

80 75 90
60 70 65
95 85 80

Output:

Average marks per student:
Student 1: 81.67
Student 2: 65.00
Student 3: 86.67

Subject-wise average:
Subject 1: 78.33
Subject 2: 76.67
Subject 3: 78.33